Luleå Technical University
School of Computer Science, Electrical and Space Engineering
Luleå, Sweden

Project Report, Submission 2
Group 16
D7043E, Data Mining

ECG HEARTBEAT CLASSIFICATION FOR EDGE AI ON THE MAX78002

Ekpenyong Esu Ekpenyong
ekpekp-5@student.ltu.se

Sreenidhi Thallam
sretha-5@student.ltu.se

Sowmya Nelakurthi
sownel-4@student.ltu.se

Examiner: Rajkumar Saini
LTU, Luleå

5th November 2026

Contents

1. Business/Application Understanding **1**
1.1. Application Scenario . . . 1
1.2. Data Mining Objective . . . 1
1.3. Project Objectives . . . 1
1.4. Success Criteria . . . 1
1.5. Constraints . . . 2
1.6. Risks and Challenges . . . 3

2. Data Understanding **3**
2.1. Dataset Description . . . 3
2.2. Target Classes . . . 4
2.3. Exploratory Data Analysis . . . 5
2.4. Data Quality . . . 9
2.5. Leakage Analysis . . . 10

3. Data Preparation **11**
3.1. Record Selection . . . 11
3.2. ECG Channel Selection . . . 12
3.3. Beat Extraction . . . 12
3.4. Signal Preprocessing . . . 13
3.5. Label Preparation . . . 14
3.6. Class Imbalance . . . 14
3.7. Data Augmentation . . . 15
3.8. Final Dataset . . . 15
3.9. Reproducibility . . . 15

4. Modeling **16**
4.1. Baseline . . . 16
4.2. Proposed Edge Model . . . 16
4.3. Training Configuration . . . 17
4.4. Model Selection . . . 18
4.5. Quantization . . . 19

5. Evaluation **20**
5.1. Clean Test Results . . . 20
5.2. Per-Class Performance . . . 20
5.3. Quantization Analysis . . . 21
5.4. Noise Robustness . . . 22
5.5. Error Analysis . . . 24
5.6. Success Criteria Review . . . 27

6. Deployment **27**
6.1. MAX78002 Compatibility . . . 27
6.2. MAX78002 Toolchain . . . 28
6.3. Deployment Evidence . . . 29
6.4. Hardware Limitations . . . 29
6.5. Proposed Hardware Validation . . . 30

7. Final Discussion **30**

8. Limitations **31**

References 33

1. Business/Application Understanding

1.1. Application Scenario

An electrocardiogram (ECG) records the electrical activity of the heart, and a cardiologist can tell from the shape and timing of each beat whether it is normal or originates from the wrong part of the heart. A day of continuous ECG contains around 100 000 beats, so manual review of long recordings is impractical. Long-term monitoring depends on software that flags abnormal beats for the clinician to review.

This project builds the first stage of such a system: a small neural network that classifies one heartbeat as normal, supraventricular ectopic, ventricular ectopic or fusion. Ectopic beats originate outside the sinus node. Supraventricular ectopic beats start above the ventricles and are often close to normal in shape. Ventricular ectopic beats start in the ventricles and typically have a wide, abnormal QRS complex. A fusion beat occurs when a normal and a ventricular activation coincide.

Automatic beat classification is used in Holter recorders, hospital telemetry, wearable patches and smartwatches, and in screening where cardiologists are scarce. The target here is the wearable case, where running the classifier on the device has clear advantages over sending the signal to a server. The ECG stays on the device, which protects privacy. No network connection is needed. The result is available within milliseconds, which allows the device to alert the wearer. Transmitting raw ECG over a radio link also consumes far more energy than running a small model locally.

Potential users are patients wearing a monitor at home, clinicians reviewing flagged beats, device manufacturers, and researchers studying how such models generalise across people. This is a course project. The model is not a medical device and no clinical validation is claimed.

1.2. Data Mining Objective

Input: A fixed-length window of 256 samples (about 0.7 seconds at 360 Hz) from a single ECG lead (MLII), cut around one annotated R-peak, filtered and normalised. The window covers 90 samples before the peak and 166 samples after it, so the model sees the P wave, the QRS complex and the T wave of one beat.

Output: One of four classes for that beat: N (normal-type), S (supraventricular ectopic), V (ventricular ectopic) or F (fusion).

Type of task: Supervised multiclass classification with four classes and a strong class imbalance.

1.3. Project Objectives

1. Distinguish N, S, V and F beats from a single-lead ECG window.
2. Generalise to people the model has never seen, using a record-independent split.
3. Stay usable when the ECG contains realistic noise from movement and muscle activity.
4. Keep the model small enough for an embedded device: well under 2 MB of weights.
5. Make the model run on the MAX78002 CNN accelerator, verified by the Analog Devices software toolchain.

1.4. Success Criteria

Accuracy alone is not a meaningful target. Almost 90 % of beats are normal, so a model that always predicts N reaches 90 % accuracy and detects nothing. The project is judged on the criteria in Table 1, all measured once on the test set DS2.

Table 1: Success criteria. All are measured once, on DS2, after the model is frozen.

Criterion	Metric	Target
Predictive performance	Macro-averaged F1 over the four classes on DS2	≥ 0.70
Minority class performance	Recall (sensitivity) for S and for F on DS2	≥ 0.50 each; V recall ≥ 0.85
Noise robustness	Drop in macro-F1 from clean ECG to 6 dB SNR	≤ 0.15
Model complexity	Number of parameters and INT8 weight size	$\leq 100\,000$ parameters; ≤ 200 kB
MAX78002 compatibility	ai8x quantisation and synthesis complete and produce C code	PASS on all four toolchain checks

In addition, the INT8 model must lose no more than 0.03 macro-F1 relative to the FP32 model, since only the INT8 model can be deployed.

1.5. Constraints

Limited memory. The MAX78002 accelerator has about 2 MB of weight memory and a few hundred kilobytes for activations. Architectures common on GPUs do not fit.

Fixed-point arithmetic. The accelerator computes with 8-bit integers. Weights and activations must be quantised, and the model has to survive that with little loss.

Class imbalance. About nine in ten beats are normal and fusion beats are below 1 % of the data. A model can ignore the rare classes and still score well on accuracy.

Few subjects. The database has 48 records from 47 people. After removing the four paced records, I train on 22 records (DS1) and test on 22 others (DS2). This is a demanding generalisation task, and the rare classes are concentrated in a few records.

Noise. Real wearable ECG contains baseline drift, muscle activity and electrode movement. The training data is comparatively clean hospital ECG.

Computational resources. Training runs on a laptop or a modest GPU, which limits the size of hyperparameter searches.

No physical board. No MAX78002 development kit is available. Deployment is verified in software only, and no measured latency, power or on-device accuracy is reported.

1.6. Risks and Challenges

Table 2: Main project risks and how they are handled.

Risk	Why is it important?	Proposed mitigation
The model cannot be synthesised for the MAX78002	A model that does not run on the chip fails the main objective regardless of its accuracy. Discovering this late leaves no time to change the architecture.	Run a tiny model through the whole ai8x toolchain before tuning accuracy, and repeat after every architecture change. Use only convolution, pooling, ReLU and one small linear layer.
Poor recall on S and F beats	S beats resemble normal beats and F beats are rare. Low recall on these classes makes the classifier of little clinical use.	Class-weighted loss and oversampling of S and F; monitor per-class recall during model selection; widen the input window if S recall stays low.
Optimistic results from data leakage	Beats from the same person are highly similar. Any overlap between training and test inflates the score and hides the inter-patient gap.	Split strictly by record (DS1/DS2), validate by record inside DS1, and block test-set evaluation in code until the model is frozen.
Large accuracy loss after INT8 quantisation	The deployed model is the INT8 one. If it is much worse than the FP32 model, the reported accuracy is not what the device delivers.	Scale inputs to the accelerator range, build with ai8x layers from the start, try post-training quantisation first, use quantisation-aware fine-tuning if needed.
Model breaks down under noise	Wearable ECG is noisier than hospital ECG. A model that only works on clean signals is not useful in practice.	Baseline-wander removal, noise augmentation during training, and a dedicated noise test at 12, 6 and 0 dB.

2. Data Understanding

2.1. Dataset Description

The main dataset is the MIT-BIH Arrhythmia Database from PhysioNet [1], the standard benchmark for beat classification, which allows comparison with published results.

Table 3: Overview of the MIT-BIH Arrhythmia Database.

Property	Value
Database	MIT-BIH Arrhythmia Database, PhysioNet, version 1.0.0
Records	48
Subjects	47 (records 201 and 202 are from the same person)
Channels	2 per record
Sampling frequency	360 Hz
Resolution	11 bits over a 10 mV range
Duration	About 30 minutes per record, about 24 hours in total
Annotations	Beat-by-beat labels on the R-peak, made by two cardiologists with disagreements resolved
Beat annotations	About 110 000
ECG leads	Modified limb lead II (MLII) is present in 46 records and is the first channel in 45 of them; the other channel is usually V1, sometimes V2, V4 or V5. Records 102 and 104 have no MLII.

The recordings were made at Beth Israel Hospital in Boston between 1975 and 1979. Twenty-three records were selected at random from routine Holter tapes and twenty-five were selected because they contain rare but clinically important rhythms, which is why the database contains a usable number of abnormal beats despite its size.

For the noise experiment we use the MIT-BIH Noise Stress Test Database from the same source [2]. It provides three half-hour recordings of pure noise, made with real electrodes on moving volunteers: baseline wander (`bw`), electrode motion artefact (`em`) and muscle artefact (`ma`). They are sampled at 360 Hz, so they can be added directly to the arrhythmia recordings.

2.2. Target Classes

The database uses about fifteen beat symbols. They are grouped into the four required classes following the AAMI EC57 standard, which is also the grouping used in most inter-patient studies. Table 4 lists the mapping and the beat counts over the 44 records used.

Table 4: Mapping from MIT-BIH beat symbols to the four target classes. Counts are produced by the preprocessing step of `notebooks/EDA.ipynb` and stored in `results/split_stats.csv`.

Class	MIT-BIH symbols	Meaning	Beats
N	`N`, `L`, `R`, `e`, `j`	Normal beat; left and right bundle branch block beats; atrial and nodal escape beats	90 125
S	`A`, `a`, `J`, `S`	Atrial premature; aberrated atrial premature; nodal premature; supraventricular premature beats	2 781
V	`V`, `E`	Premature ventricular contraction; ventricular escape beat	7 009
F	`F`	Fusion of ventricular and normal beat	803

Excluded annotations. Three kinds of labels are dropped. The preprocessing script counts every dropped beat, so the exclusions are reproducible.

- **Paced beats** (`/`) and **fusion of paced and normal beats** (`f`). These come from patients with pacemakers. The four paced records (102, 104, 107 and 217) are excluded entirely, as is standard practice, because the pacemaker spike is an artificial signal unrelated to the heart's own rhythm.

- **Unclassifiable beats** (`Q`). The annotators could not assign a class, so they cannot serve as training targets.
- **Non-beat annotations**, such as rhythm changes (`+`), noise markers (`~`) and signal-quality flags. These describe the recording, not a heartbeat, and are never counted as beats.

Excluding the paced records leaves 44 records, split evenly into DS1 and DS2 as described in Section 3.1..

2.3. Exploratory Data Analysis

The full analysis is in `notebooks/EDA.ipynb`. The figures below summarise the main findings.

The raw signal. Figure 1 shows ten seconds of record 100, both channels. The MLII channel has a tall, sharp QRS complex on every beat, which makes it the preferred lead for beat analysis. The V1 channel shows the same beats with a smaller, partly inverted QRS.

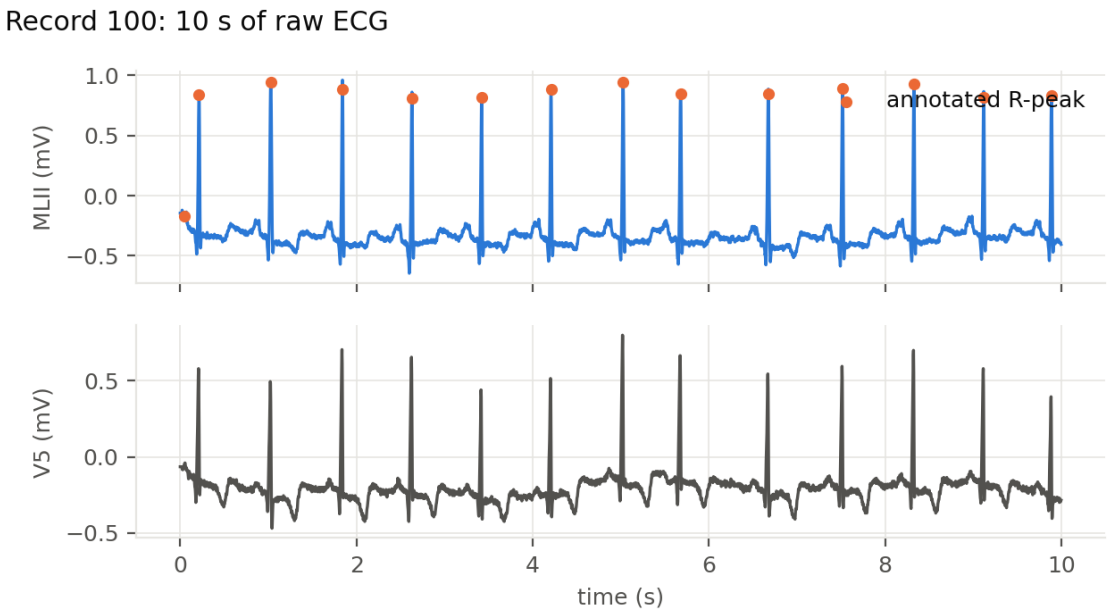


Figure 1: Ten seconds of raw ECG from record 100.

One beat per class. Figure 2 shows a typical beat of each class, aligned on the R-peak. N and S beats are difficult to separate by shape; the main difference is timing, since the S beat arrives earlier than expected. The V beat has a wide QRS and no normal P wave. The F beat has an intermediate shape between N and V.

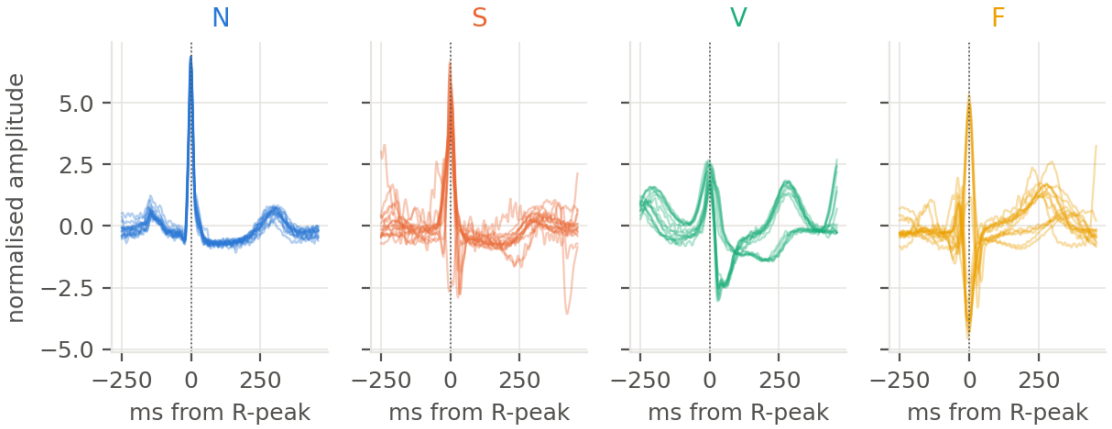


Figure 2: Example heartbeat of each class, in the 256-sample window aligned on the R-peak (Section 3.3.).

Each class in its rhythm. Figure 3 shows one beat of each class in four seconds of signal, with the beats before and after it. This shows what a single window cannot. The S beat looks almost like an N beat, but it comes earlier than the rhythm predicts and is followed by a longer pause. The V beat is wide and tall, has no P wave in front of it, and is followed by a long pause. The F beat comes at the normal time, but its shape is partly normal and partly ventricular.

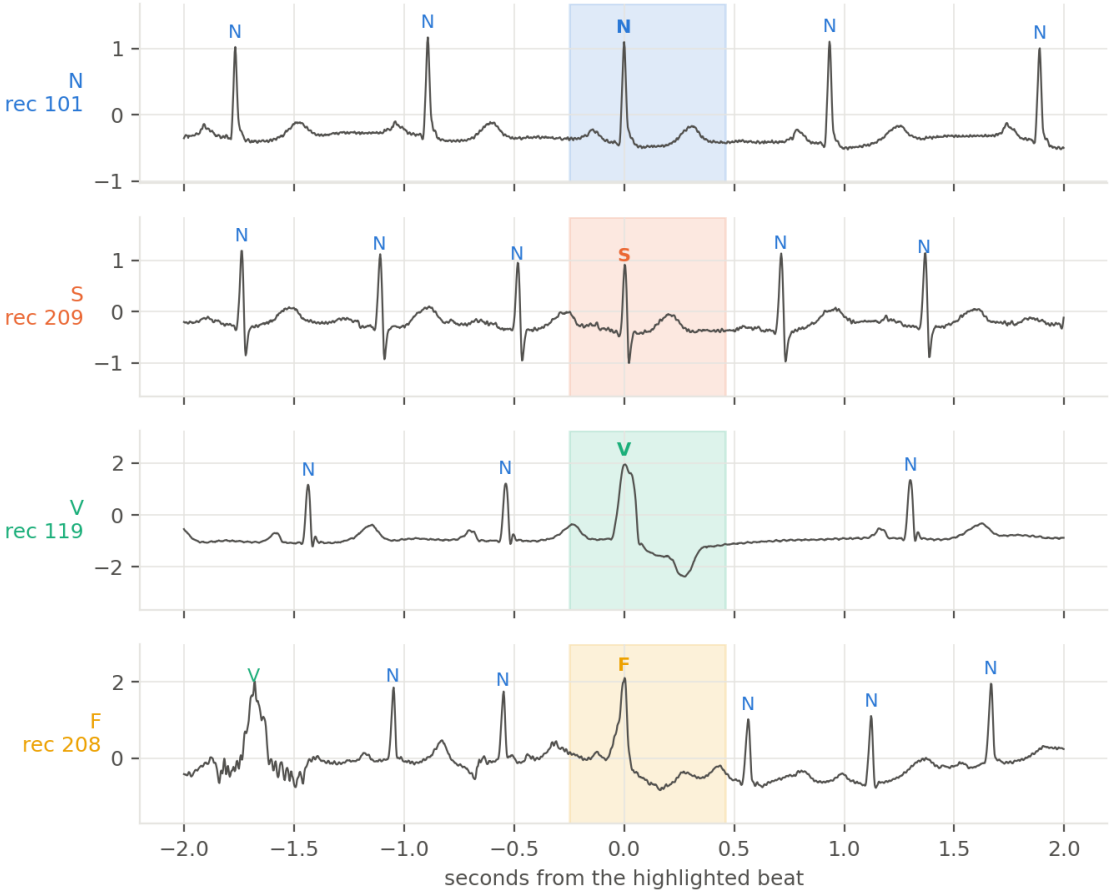


Figure 3: One beat of each class (bold label, shaded 256-sample window) with its neighbouring beats. Raw MLII signal in mV. All examples are from DS1 records.

Class distribution. Figure 4 shows how uneven the classes are. Normal beats make up close to 90 % of the data, V beats are the largest minority, S beats are about 2–3 % and F beats well under 1 %. This imbalance motivates several decisions in Section 3..

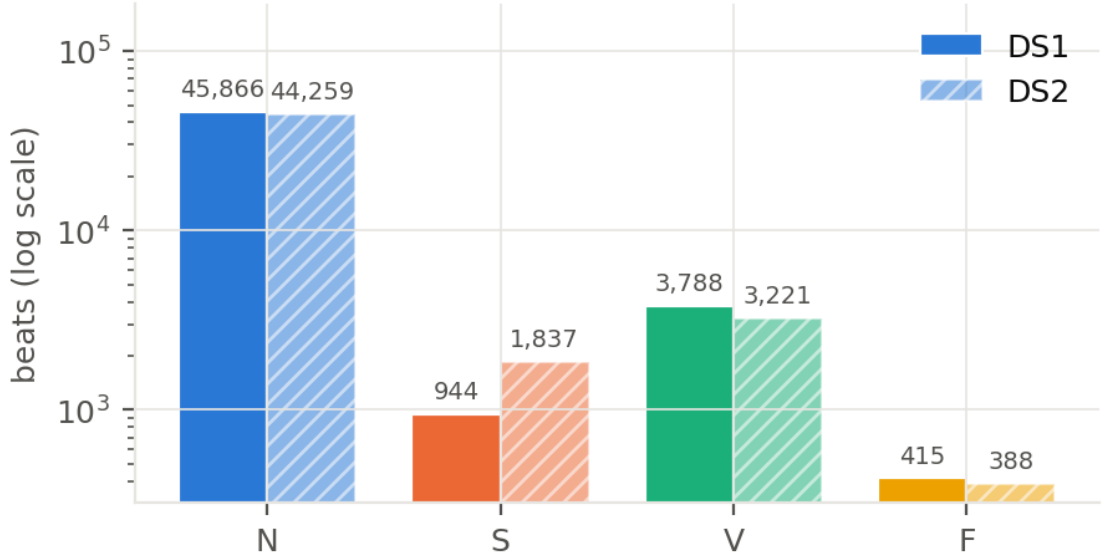


Figure 4: Number of beats per class in DS1 and DS2 (logarithmic scale).

Beats per record. Figure 5 shows the class composition of each record, and Table 5 gives the exact counts. The rare classes are not spread evenly. Record 232 alone holds 1 382 of the 2 781 S beats, about half of them, and records 209 and 222 add another 592. For F beats it is even more extreme: records 208 and 213 hold 735 of the 803 F beats. Record 208 is in DS1 and record 213 is in DS2, so the model learns F mostly from one person and is tested mostly on another. Performance on these classes therefore depends heavily on a few individual patients.

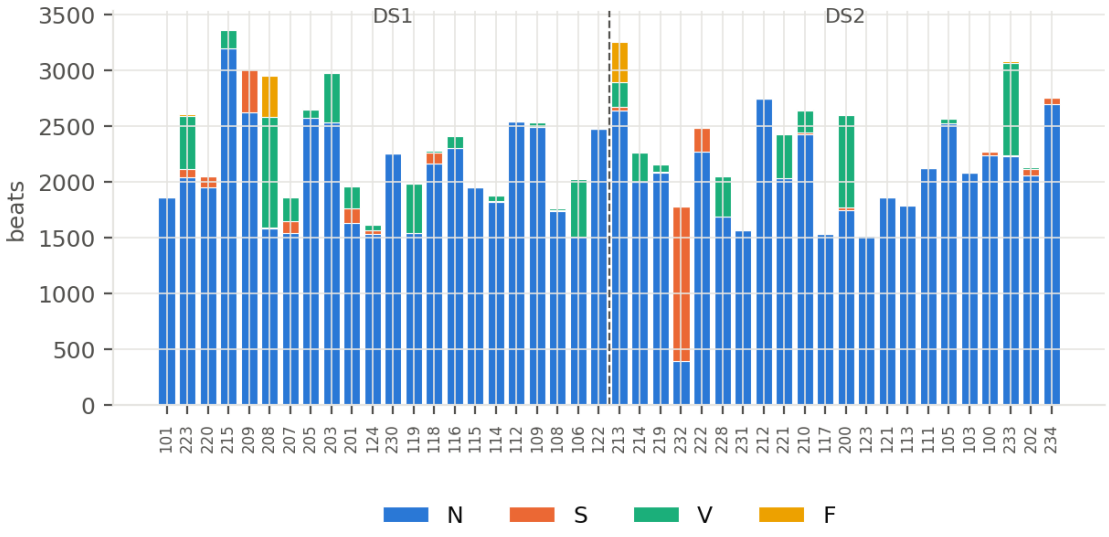


Figure 5: Class composition of each record.

Table 5: Beats per class in each record. DS1 records are on the left, DS2 records on the right.

DS1 record	N	S	V	F	DS2 record	N	S	V	F
101	1 860	3	0	0	100	2 239	33	1	0
106	1 507	0	520	0	103	2 082	2	0	0
108	1 740	4	17	2	105	2 526	0	41	0
109	2 492	0	38	2	111	2 123	0	1	0
112	2 537	2	0	0	113	1 789	6	0	0
114	1 820	12	43	4	117	1 534	1	0	0
115	1 953	0	0	0	121	1 861	1	1	0
116	2 302	1	109	0	123	1 515	0	3	0
118	2 166	96	16	0	200	1 743	30	826	2
119	1 543	0	444	0	202	2 061	55	19	1
122	2 476	0	0	0	210	2 423	22	195	10
124	1 536	31	47	5	212	2 748	0	0	0
201	1 635	128	198	2	213	2 641	28	220	362
203	2 529	2	444	1	214	2 003	0	256	1
205	2 571	3	71	11	219	2 082	7	64	1
207	1 543	107	210	0	221	2 031	0	396	0
208	1 586	2	992	373	222	2 274	209	0	0
209	2 621	383	1	0	228	1 688	3	362	0
215	3 195	3	164	1	231	1 568	1	2	0
220	1 954	94	0	0	232	398	1 382	0	0
223	2 045	73	473	14	233	2 230	7	831	11
230	2 255	0	1	0	234	2 700	50	3	0

Signal amplitude. Figure 6 shows the distribution of R-peak amplitude per record. Amplitudes vary by a factor of several between records because electrode placement and skin contact differ between subjects. Each beat is therefore normalised individually rather than with one global scale.

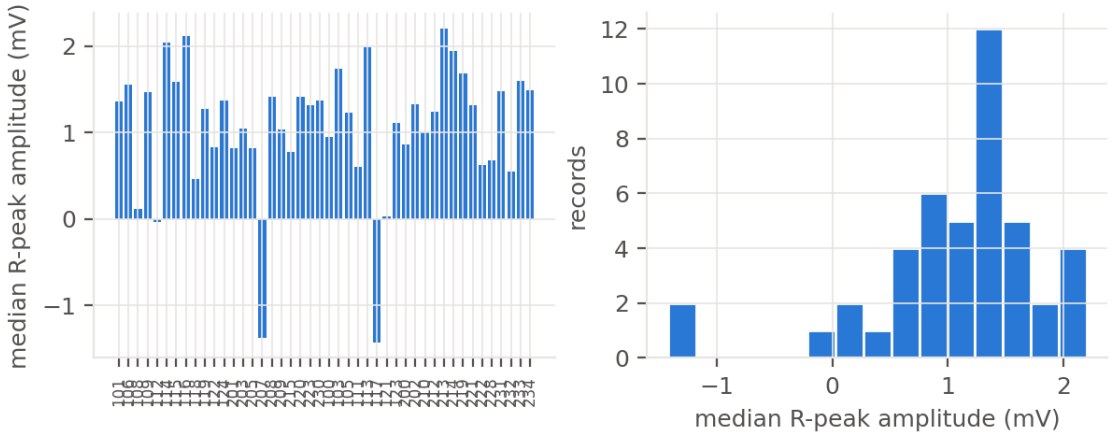


Figure 6: Distribution of R-peak amplitude across records.

Noisy ECG. Figure 7 shows segments from records 203 and 207, which are known to be noisy,

together with the three noise types from the Noise Stress Test Database. Baseline wander is a slow drift, muscle artefact is high-frequency noise, and electrode motion produces abrupt transients that can resemble beats.

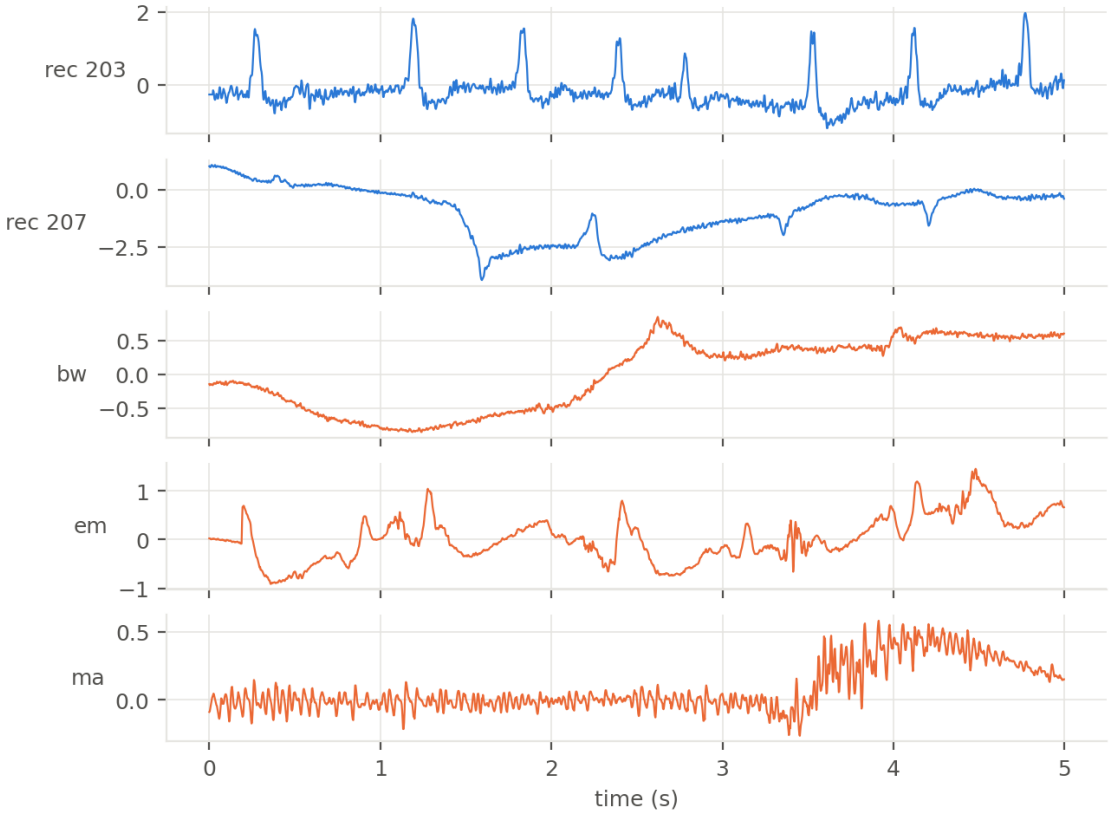


Figure 7: Examples of noisy ECG and of the three noise types used in the robustness test.

2.4. Data Quality

Missing data. The recordings have no missing samples. Every record is complete at 360 Hz.

Extreme amplitudes. A few records contain short bursts where the signal saturates at the limits of the 10 mV range, usually during electrode movement. These segments are rare. Per-beat normalisation and clipping limit their effect on the input.

Corrupted segments. The annotators marked noisy stretches with the ~ symbol. Table 6 lists the records with the most of these marks. Record 105 has the most by far, followed by 203, 200 and 108. The noisy records are spread over both splits, so neither the training set nor the test set is much cleaner than the other. The beats inside these segments are kept, since a wearable will encounter the same kind of signal, but they are a likely source of label noise.

Table 6: Records with the most noise annotations (~).

Record	Split	Noise marks (~)
105	DS2	88
203	DS1	57
200	DS2	43
108	DS1	41
232	DS2	35
106	DS1	30
215	DS1	30
208	DS1	24
228	DS2	20
209	DS1	19

Class imbalance. Described above; it is the main data-quality issue.

Differences between recordings. Heart rate, beat shape, amplitude and noise level all vary from record to record. Some records show only normal rhythm, others show bundle branch block on every beat, and a few contain long runs of ventricular beats. Because of this variation, results on a small number of records can be misleading.

Differences between channels. Table 7 shows which lead is stored in each channel. MLII is the first channel in 43 of the 44 records used, but in record 114 the two channels are stored in the opposite order. Selecting the lead by name rather than by column index avoids reading the wrong signal. The second channel also changes between records, so only MLII is the same in every record. Records 102 and 104 have no MLII; both are paced records and are already excluded.

Table 7: Lead stored in each channel, over the 44 records used.

Channel 1	Channel 2	Records	Record IDs
MLII	V1	38	101, 105, 106, 108, 109, 111, 112, 113, 115, 116, 118, 119, 121, 122, 200, 201, 202, 203, 205, 207, 208, 209, 210, 212, 213, 214, 215, 219, 220, 221, 222, 223, 228, 230, 231, 232, 233, 234
MLII	V2	2	103, 117
MLII	V5	2	100, 123
MLII	V4	1	124
V5	MLII	1	114

Possible noisy regions. Baseline wander is present to some degree in almost every record. It is removed in preprocessing rather than detected and skipped.

2.5. Leakage Analysis

Data leakage occurs when information from the test set reaches the model during training, so the test score no longer measures generalisation.

A record is thirty minutes of ECG from one person. Its roughly 2 000 beats share the same heart, electrodes and skin contact, so they are highly similar. If all beats were pooled, shuffled and split 80/20, nearly every test beat would have a near-duplicate in the training set. A model could then memorise each patient's waveform and exceed 99 % test accuracy without learning what distinguishes the classes. Studies using such splits report accuracies of this kind, which fall to 80–90 %

when the same models are tested on unseen subjects. A wearable device only sees unseen subjects, so the shuffled score does not measure the quantity of interest.

Three measures prevent this. First, the data is split by record using the standard inter-patient partition: 22 records form the development set DS1 and 22 different records form the test set DS2, with no record in both. The partition has one known weakness: records 201 and 202 come from the same subject, and the standard split places 201 in DS1 and 202 in DS2. The partition is kept as published so that results remain comparable with the literature, and this is noted as a limitation. Second, the training/validation split inside DS1 is also by record, so model selection uses subjects the model was not trained on. Third, the split is enforced in code: the split-building function rejects a validation record that is not in DS1 and checks that no record is listed twice, and the evaluation script refuses to run on DS2 without an explicit `-final` flag. DS2 is used once per final model, after all design decisions are fixed.

The same rule applies to the noise experiment. Noise is added to DS2 only at evaluation time and the noisy signals are never used for training. Noise used as training augmentation is added to DS1 records only.

3. Data Preparation

All steps in this section are implemented in the `ecg.data` package and run from section 0.2 of `notebooks/EDA.ipynb`. Every tunable number comes from `configs/preprocess.yaml`, so the whole pipeline can be rerun from the raw PhysioNet files with one command.

3.1. Record Selection

The split follows the inter-patient partition of de Chazal et al. [3], as recommended by the assignment. The four paced records are excluded.

Table 8: Record assignment. Training and validation together form DS1.

Split	Records	Record numbers
Training (DS1 minus validation)	17	101, 106, 108, 109, 112, 114, 115, 116, 118, 119, 122, 203, 207, 208, 209, 215, 230
Validation (subset of DS1)	5	124, 201, 205, 220, 223
Test (DS2)	22	100, 103, 105, 111, 113, 117, 121, 123, 200, 202, 210, 212, 213, 214, 219, 221, 222, 228, 231, 232, 233, 234

The validation records were chosen after inspecting the per-record class counts (Figure 5). A first choice that placed record 208 in validation left only 20 fusion beats for training, because record 208 alone holds 372 of the 415 F beats in DS1. The choice keeps records 208 and 209, which hold most of the F and S beats respectively, in training. Validation is used for early stopping, hyperparameter choice and model selection. DS2 is not used until the final model is fixed.

Revision in Submission 2. Submission 1 used records 118, 124, 201, 203 and 205 for validation. The first training runs of the modeling stage showed two problems with that choice, and CRISP-DM asks for exactly this kind of return to an earlier stage. First, records 118 and 124 are the only two DS1 records whose beats show right bundle branch block (they contain 2 166 and 1 531 `R` beats, which map to class N). With both in validation, the training set had almost no beats of that shape, and DS2 contains two more such records (212 and 231). Second, record 203 is the noisiest record in the database (Table 6) and made up a quarter of the validation beats. Its noisy N beats were predicted as V or F more and more as training went on, so the validation score fell from the first epoch while the other four records improved; early stopping then always picked an almost untrained network. The final split therefore moves 118 and 203 into training, where the

first adds bundle branch block beats and the second adds realistic noise, and takes 220 and 223 into validation instead. Validation now holds 124 (bundle branch block, a few F), 201 (S and V), 205 (V and F), 220 (S) and 223 (S, V and F). The change was made before any DS2 evaluation, using validation results only. Table 10 gives the resulting counts: validation holds 329 S beats but only 32 fusion beats, which is a limitation of the dataset rather than of the split; no assignment of DS1 records gives both training and validation a large F class.

3.2. ECG Channel Selection

A single lead, MLII, is used. It is present in every record used, it is the lead the annotators worked from, and it shows the QRS complex most clearly. A single lead also matches the wearable setting, where a patch or watch typically records one channel.

The lead is selected by its name in the record header rather than by position, because in record 114 MLII is stored as the second channel. If a record has no MLII the script raises an error rather than substituting another lead. This only affects records 102 and 104, which are excluded.

3.3. Beat Extraction

No beat detector is used. R-peak positions are taken from the expert annotation files, which is the standard approach for this database and keeps the project focused on classification.

Around each annotated R-peak a window of 90 samples before and 166 samples after the peak is extracted, giving a fixed input length of 256 samples, about 0.71 seconds at 360 Hz. The window is asymmetric: 90 samples (250 ms) before the peak covers the P wave, and 166 samples (460 ms) after it covers the T wave at normal heart rates. A length of 256 is a power of two, which simplifies the memory layout on the MAX78002.

Revision in Submission 2. Model selection (Section 4.4.) also tried a wider window of 256 samples before and 256 after the peak (512 samples, 1.42 s), kept in `configs/preprocess_w512.yaml`. It usually contains the previous and the next R-peak, so the network can see whether a beat came early, which is what separates an S beat from an N beat. That window doubled the S recall on the validation records and became the final input. All other preprocessing steps are unchanged.

Figure 8 shows how one window is cut. The top panel shows the annotated R-peaks from the `.atr` file, each with its beat symbol and sample index. The bottom panel shows the window that is cut around one of them. The R-peak always sits at index 90, so every beat in the dataset is lined up in the same way.

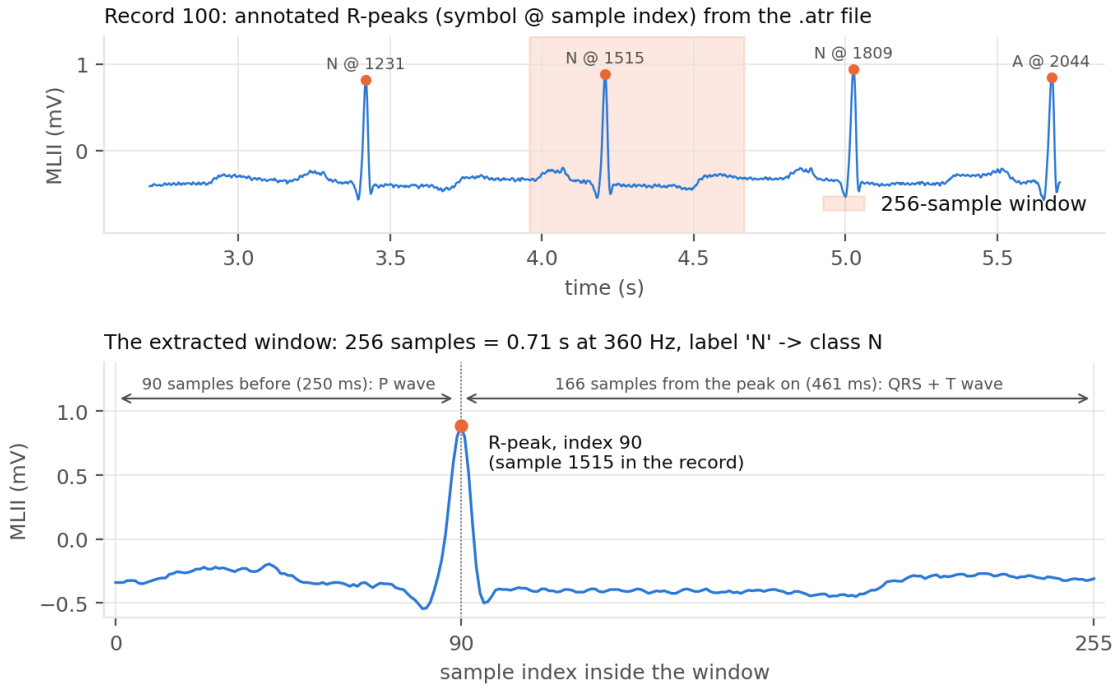


Figure 8: Beat extraction on record 100. Top: raw MLII signal with the expert annotations and one 256-sample window shaded. Bottom: the extracted window, before filtering and normalisation.

Beats whose window extends past the start or end of the recording are dropped, which affects at most one or two beats per record.

The window is described here because the steps that follow work on it. In the pipeline, though, the whole signal is filtered first and cut afterwards, as the next subsection shows.

3.4. Signal Preprocessing

The operations are applied in this order:

`Raw MLII signal` → `baseline removal` → `low-pass filter` → `segmentation` → `per-beat normalisation` → `clipping` → `model input (1×256)`

Baseline removal. Two cascaded median filters with windows of 200 ms and 600 ms estimate the slow baseline, which is then subtracted from the signal. Breathing and body movement shift the baseline by more than the height of a QRS complex, so without this step identical beats at different points of the drift would look different to the model. A median filter removes the drift without distorting the QRS and is cheap enough to run on the microcontroller's CPU.

Low-pass filter. A zero-phase Butterworth filter with a 40 Hz cut-off removes muscle noise and mains interference above the frequency range of the ECG itself. This step is optional in the configuration so that its effect can be measured in the noise experiment.

Segmentation. The filtered signal is cut into 256-sample windows as described above. Segmentation follows filtering so that the filters operate on a continuous signal and produce no edge effects inside a beat.

Per-beat normalisation. Each window is standardised to zero mean and unit standard deviation. Amplitude varies by a factor of several between subjects and electrode placements and carries no class information. Normalisation is applied per beat rather than per record because a wearable does not know the record statistics in advance.

Clipping. Values are clipped to the range $[-8, 8]$ standard deviations. This bounds the input, which 8-bit quantisation requires, and prevents a single electrode-motion spike from dominating the window. The limit was first set to 4, but inspection of the processed beats showed that R-peaks typically reach 5 to 7 standard deviations, so a limit of 4 flattened the peak on 74 % of beats and did so far more often for N and S beats than for V beats. A limit of 8 leaves all ordinary beats untouched. In the final step before the model the bounded range is mapped to the accelerator's input range of $[-1, 1)$.

RR intervals. Alongside each window, the time to the previous and next annotated beat is stored, together with the ratio of the previous interval to the mean of the ten intervals before it. These values are capped at 3 seconds, because unannotated episodes such as the 100-second ventricular flutter in record 207 would otherwise produce meaningless intervals. They are not part of the CNN input; the baseline uses them.

3.5. Label Preparation

The mapping from annotation symbol to class is a single dictionary in `src/ecg/constants.py`, reproduced here in full. Any symbol not listed is either an excluded beat or a non-beat annotation and is dropped with a count.

Table 9: Complete label mapping.

Symbol	PhysioNet description	Class	Index
N	Normal beat	N	0
L	Left bundle branch block beat	N	0
R	Right bundle branch block beat	N	0
e	Atrial escape beat	N	0
j	Nodal (junctional) escape beat	N	0
A	Atrial premature beat	S	1
a	Aberrated atrial premature beat	S	1
J	Nodal (junctional) premature beat	S	1
S	Supraventricular premature beat	S	1
V	Premature ventricular contraction	V	2
E	Ventricular escape beat	V	2
F	Fusion of ventricular and normal beat	F	3
/, f, Q	Paced, fusion of paced and normal, unclassifiable	excluded	–
others	Rhythm, noise and signal-quality annotations	not a beat	–

3.6. Class Imbalance

In the training set roughly 90.0 % of beats are normal, 1.5 % are S, 7.5 % are V and 1.0 % are F, so the ratio between the most and least common class is about 95 to one. With plain cross-entropy a network converges towards predicting N almost always, because that minimises the loss.

Two measures are planned. First, a **class-weighted cross-entropy loss**, with each class weighted by the inverse square root of its frequency; the square root moderates the weights so that rare classes are emphasised without destabilising training. Second, **oversampling of the rare classes** in the training sampler, with each beat drawn in proportion to the inverse square root of its class frequency, so that each batch contains more minority beats than their natural share.

Undersampling normal beats would discard most of the data. Focal loss is kept as a configurable alternative for model selection, and no balancing is included as a negative control. The final choice

is made on validation macro-F1, not on the test set. Section 4.4. reports the comparison: the two measures together turned out to be too strong for unseen records, and oversampling alone was selected.

3.7. Data Augmentation

Augmentation is applied on the fly to training beats only. Validation and test beats are never augmented, so the validation score remains an unbiased estimate. Each transformation is a small random change that preserves the class:

- **Amplitude scaling** by a random factor between 0.8 and 1.2, since amplitude carries no class information.
- **Time shift** of up to ± 5 samples, so the model tolerates small errors in R-peak alignment.
- **Baseline wander**, a low-frequency sinusoid added to the window, to prepare the model for imperfect baseline removal.
- **Gaussian noise** with a standard deviation of about 0.02 after normalisation.
- **Real ECG noise** from the Noise Stress Test Database at a signal-to-noise ratio of 6 dB or better, as an optional setting. (Submission 2 note: this option was not used in the final training runs for lack of time; Section 5.4. discusses it as the first improvement to try.)

Beats are not stretched or compressed in time. QRS width is one of the main features separating V from N beats, so changing it would corrupt the labels.

3.8. Final Dataset

Table 10: Beat counts per split with the revised validation records, produced by section 0.2 of `notebooks/EDA.ipynb` and stored in `results/split_stats.csv`.

Split	Records	N	S	V	F	Total
Train	17	36 112	615	2 999	382	40 108
Validation	5	9 736	329	789	32	10 886
Test (DS2)	22	44 240	1 837	3 220	388	49 685

3.9. Reproducibility

Table 11: Settings needed to reproduce the preprocessing. All are stored in `configs/preprocess.yaml`.

Item	Value
Python	3.10
Key packages	wfdb 4.3.1, NumPy 2.2.6, SciPy 1.15.3, scikit-learn 1.7.2, PyTorch 2.14.1; exact versions in `requirements.txt`
Random seed	42, applied to Python, NumPy and PyTorch
Sampling frequency	360 Hz, no resampling
Lead	MLII, selected by name
Window	90 samples before the R-peak, 166 after; input size 1×256 (Submission 1). Final model: 256 before, 256 after; 1×512 (`configs/preprocess_w512.yaml`)
Baseline removal	Cascaded median filters, 200 ms and 600 ms
Low-pass filter	Butterworth, 40 Hz cut-off, zero-phase
Normalisation	Per-beat z-score, clipped to $[-8, 8]$
Data source	PhysioNet `mitdb` 1.0.0 and `nstdb` 1.0.0, downloaded with the `wfdb` package

Running `make eda` from a clean checkout (section 0 of `notebooks/EDA.ipynb`) downloads the data and regenerates every array and every number in this section. Section 2 of `notebooks/MODELING.ipynb` builds the 512-sample variant the same way.

4. Modeling

All models in this section are trained on the 17 training records and compared on the 5 validation records of Table 8. DS2 is not read until Section 5.. Everything runs from `notebooks/MODELING.ipynb`, which calls the package `ecg` (`src/ecg/datasets`, `models`, `training`); the settings are in `configs/model.yaml`, `baselines.yaml` and `sweep.yaml`, and every training run is one row of `results/experiments.csv`.

4.1. Baseline

Two classical models serve as reference points. Both receive the flattened beat window (the same 256 values the first CNN sees) plus the three RR-interval features of Section 3.3.: the time since the previous beat, the time to the next beat, and the ratio of the first to the local mean. The RR features are the standard hand-made timing cue that separates an early S beat from an N beat, so the baselines are not weak strawmen: on timing they know more than a single-beat CNN.

- **Logistic regression** on standardised features, with balanced class weights.
- **Random forest** with 200 trees, at least two beats per leaf, and balanced class weights.

Table 12: Baselines on the validation records. Both see the flattened beat plus the three RR features. Params: coefficients for the logistic regression, tree nodes for the forest.

Model	Params	Accuracy	Macro-F1	S prec.	S recall	V recall	F recall
Logistic regression	1 040	0.894	0.462	0.254	0.319	0.535	0.125
Random forest	150 574	0.933	0.430	0.933	0.043	0.529	0.000

Table 12 shows the validation scores. The logistic regression reaches a macro-F1 of 0.46, the random forest 0.43. The forest is more accurate (0.93) but finds almost no S beats (recall 0.04) and no F beats, even though it has the RR features: with 90 % normal beats, the trees still favour the majority class on unseen records. The logistic regression trades accuracy for a better S recall (0.32). Both are far below the success criterion of 0.70, which sets the bar the CNN has to clear.

4.2. Proposed Edge Model

The final network is a 1D CNN with four fused convolution blocks and one fully connected output layer (Table 13). It takes one channel of 512 samples, the wide beat window of Section 3.3., scaled to $[-1, 1)$.

Table 13: Layers of the final CNN as the accelerator runs them. Each row is one fused layer; the pool, when present, runs before the convolution.

Layer	Input	Operation	Output	Parameters
conv1	1 x 512	Conv1d k=5 p=2, ReLU	16 x 512	96
conv2	16 x 512	MaxPool 2, Conv1d k=5 p=2, ReLU	32 x 256	2 592
conv3	32 x 256	MaxPool 4, Conv1d k=3 p=1, ReLU	64 x 64	6 208
conv4	64 x 64	MaxPool 4, Conv1d k=3 p=1, ReLU	64 x 16	12 352
fc	1024 (flattened)	Linear	4	4 100
Total				25 348

Each block is one accelerator layer: an optional max-pool, a convolution with zero padding, and a ReLU. The pools halve the length after the first block, then quarter it twice, so the 512 input samples become 16 positions with 64 channels, which are flattened into 1 024 values for the output layer. Kernel lengths 5 and 3 with paddings 2 and 1 keep the length through each convolution. BatchNorm after every convolution (352 extra parameters) helped the wide-window model train in the sweep; it is folded into the convolution weights before quantisation and is absent from the deployed network, which has 25 348 weights and biases.

The architecture is appropriate for Edge-AI for four reasons. First, it uses only operations the MAX78002 accelerator executes natively (1D convolution with kernels up to 9, pooling fused into the convolution, ReLU, one small fully connected layer), so no operator falls back to the Cortex-M4 core. Second, it is small: 25 348 parameters are about 25 kB in INT8, 1 % of the chip's weight memory and a quarter of the 100 000-parameter budget of Section 1.4. Third, its activations are small too; the largest intermediate tensor is 16×512 bytes. Fourth, the input is a fixed-size tensor, which is what the accelerator's data memory expects. Transformers, recurrent layers, attention and large residual networks were not considered, because they either use unsupported operators or do not fit; the course text excludes them for the same reasons.

4.3. Training Configuration

Table 14: Training configuration of the final model.

Setting	Value
Input	1×512 samples, scaled to $[-1, 1)$
Optimizer	Adam, weight decay 0
Learning rate	0.001, cosine decay over 60 epochs
Batch size	128
Epochs	at most 60; stopped after 28, best epoch 18
Loss	cross-entropy
Class weighting	none
Oversampling	sampler weight (1 / class frequency) to the power 0.5
Early stopping	patience 10 epochs on validation macro-F1
Augmentation	amplitude scaling [0.8, 1.2], time shift ± 5 samples, baseline wander up to 0.3, Gaussian noise $\sigma = 0.02$
Dropout	0.000
Random seed	42

The loss is plain cross-entropy; the class imbalance is handled by the sampler, which draws each beat with probability proportional to the inverse square root of its class frequency, so about a third of every batch is S, V or F while N still dominates. Section 4.4. explains why this was preferred to class weights. Augmentation is applied on the fly to training beats only: amplitude scaling, a time shift of up to five samples, a slow sine as baseline wander, and light Gaussian noise. The learning rate follows a cosine decay and early stopping keeps the epoch with the best validation macro-F1 (Figure 9). All runs use seed 42 for Python, NumPy and PyTorch.

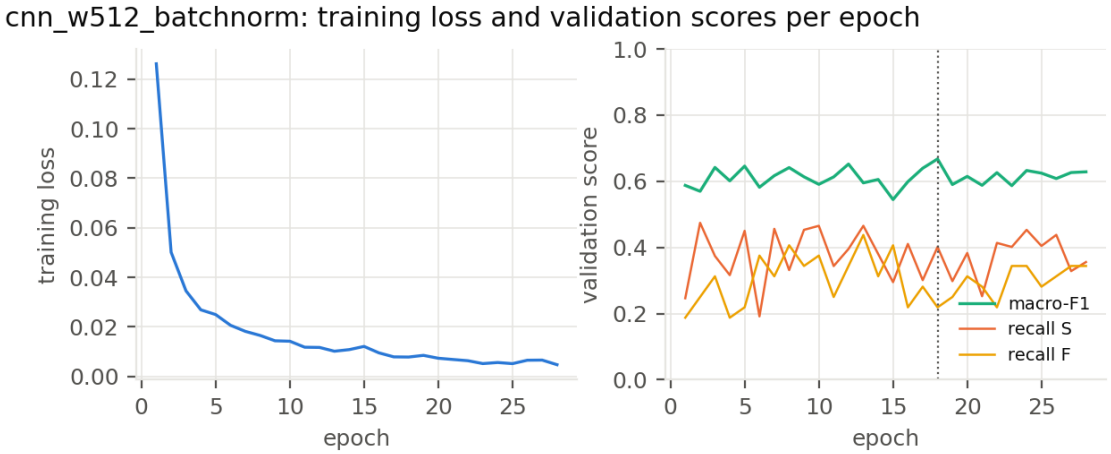


Figure 9: The final run: training loss (left) and validation macro-F1 and S/F recall per epoch (right). The dotted line marks the epoch whose weights were kept.

4.4. Model Selection

Model selection was a staged sweep (`configs/sweep.yaml`), always scored by macro-F1 on the validation records. Each stage starts from the best run so far, so a later stage only has to vary what it is about. Table 15 lists every run.

Table 15: Model-selection sweep on the validation records, best first. Window is the input length in samples.

Run	Stage	Windo	Param	Best epoch	Accur	Macro F1	S prec.	S re-call	V re-call	F re-call
cnn_w512_batchnorm	3	512	25 700	18	0.965	0.667	0.846	0.401	0.840	0.219
cnn_w512_mild_balance	3	512	25 348	35	0.951	0.614	0.616	0.444	0.802	0.281
cnn_w512_large	3	512	39 300	28	0.949	0.613	0.478	0.429	0.848	0.344
cnn_w512_small	3	512	7 428	6	0.954	0.598	0.567	0.362	0.835	0.219
cnn_w512_no_aug	3	512	25 348	9	0.946	0.596	0.454	0.453	0.788	0.219
cnn_w512	2	512	25 348	4	0.947	0.591	0.472	0.465	0.736	0.156
cnn_oversample_only	1	256	25 348	8	0.955	0.569	0.441	0.237	0.929	0.156
cnn_mild_balance	1	256	25 348	6	0.946	0.564	0.471	0.219	0.923	0.312
cnn_no_aug	2	256	25 348	5	0.938	0.562	0.544	0.280	0.833	0.250
cnn_batchnorm	2	256	25 700	20	0.951	0.557	0.416	0.286	0.920	0.031
cnn_small	2	256	7 428	8	0.953	0.555	0.508	0.195	0.901	0.156
cnn_large	2	256	39 300	22	0.943	0.554	0.249	0.222	0.961	0.156
cnn_regularised	2	256	25 348	10	0.953	0.550	0.480	0.182	0.932	0.094
cnn_weights_only	1	256	25 348	7	0.950	0.548	0.583	0.182	0.905	0.156
cnn_focal	1	256	25 348	7	0.952	0.546	0.592	0.185	0.901	0.094
cnn_plain_ce	1	256	25 348	26	0.963	0.544	0.529	0.137	0.962	0.031
cnn_weights_oversample	1	256	25 348	17	0.937	0.536	0.253	0.188	0.954	0.156

Stage 1: class imbalance, with the 256-sample window and the architecture of `configs/model.yaml`. Six runs compared the plan of Section 3.6 (weighted loss plus oversampling, both with power 0.5), each measure alone, a milder version (power 0.25 for both), focal loss [4], and no balancing as a

negative control. Oversampling alone won (macro-F1 0.569). The combination planned in Submission 1 was the worst run of the stage (0.536): weights and oversampling together give the rare classes about the same effective share as the normal class, and the network then predicts S and F far too often on unseen records, which costs more precision than it gains in recall. Plain cross-entropy had the highest accuracy (0.963) but the lowest S recall (0.14), the expected failure mode of ignoring the imbalance.

Stage 2: architecture, regularisation, augmentation and window, on top of oversampling alone. A smaller network (channels 8, 16, 32, 32; 7 428 parameters) and a larger one (32, 64, 64, 64; 39 300 parameters) scored within 0.015 of the base network, so model size is not the bottleneck. Dropout with weight decay, BatchNorm and removing augmentation changed little. The wide 512-sample window was the only change that mattered: macro-F1 rose from 0.569 to 0.591 and S recall from 0.24 to 0.47, because the window now shows the previous beat and the network can see that an S beat arrives early. This is the single most important finding of the sweep, and it is a return to the data-preparation stage, documented in Section 3.3..

Stage 3: around the stage-2 winner. With the wide window, BatchNorm now helped clearly (0.667, the best run overall, with accuracy 0.965, V recall 0.84 and S recall 0.40), while mild class weights, no augmentation, and the small and large variants stayed between 0.595 and 0.614. The small wide-window network keeps 0.598 with a quarter of the weights, which Section 5.4. uses for the robustness-versus-size comparison.

The final architecture was therefore selected because it had the highest validation macro-F1 and also the highest accuracy of all runs, and it did so with a network that passes the accelerator checks unchanged. F recall stayed low in every run (at most 0.34, on 32 validation beats), which Section 5.2. returns to.

4.5. Quantization

The deployed model must run with 8-bit weights and 8-bit activations. Two routes were compared on the validation records (Table 16); the scores come from the ai8x integer simulation of the quantised checkpoints, which Section 6.3. shows to be bit-exact with the synthesiser.

Table 16: FP32 and INT8 versions of the final model on the validation records. INT8 scores come from the ai8x integer simulation of the quantised checkpoint.

Model	Accuracy	Macro-F1	N recall	S recall	V recall	F recall
FP32	0.965	0.667	0.997	0.401	0.840	0.219
INT8, post-training (PTQ)	0.931	0.393	0.997	0.000	0.544	0.000
INT8, after QAT	0.959	0.659	0.994	0.359	0.806	0.344

Post-training quantisation (PTQ). `quantize.py` was applied directly to the FP32 weights after BatchNorm folding. The result lost most of the signal: macro-F1 fell from 0.667 to 0.39, S recall to zero. The cause is not the 8-bit weights but the activations. The accelerator clamps every layer output to $[-1, 1)$ in 8-bit steps, while the FP32 network had learned activations several times larger, so most of its dynamic range was cut off.

Quantisation-aware training (QAT) was therefore needed. Starting from the FP32 weights, BatchNorm is folded, the range of every layer's activations is measured on training data and turned into a power-of-two scale per layer (the activation-threshold step of `ai8x`), and the network is fine-tuned for 15 epochs at a low learning rate with the `ai8x` layers in QAT mode: weights rounded to 8 bits and activations clamped in the forward pass, with straight-through gradients [5]. After `quantize.py` the INT8 model reaches a validation macro-F1 of 0.659, within 0.01 of the FP32 model, and keeps the S, V and F recall. QAT thus removed almost all of the quantisation loss, at the cost of one short extra training run, and the QAT model is the one deployed and evaluated

on DS2 in Section 5.. Without the activation-threshold step, a first QAT attempt recovered only to 0.59, which shows that the per-layer scaling, not the fine-tuning alone, is what carries the small activations through the 8-bit clamp.

5. Evaluation

DS2 was read for the first time for this section, after the FP32 model, the INT8 model and the baselines were frozen on the validation records. Every model was scored once. Each model sees DS2 built with its own window (256 samples for the baselines, 512 for the CNN); the records and the annotated beats are the same, only the number of beats dropped at the record edges differs by a few. Section 5 of `notebooks/MODELING.ipynb` writes every number below to `results/`. The INT8 scores come from the ai8x integer simulation of the quantised checkpoint, which Section 6.3. shows to match the synthesiser bit for bit.

5.1. Clean Test Results

Table 17: Clean DS2 test results. The parameter count of the random forest is its number of tree nodes.

Metric	Logistic regression	Random forest	FP32 CNN	INT8 CNN (QAT)
Accuracy	0.782	0.935	0.921	0.916
Macro-F1	0.484	0.461	0.505	0.493
Model parameters	1 040	150 574	25 700	25 348

The CNN beats both baselines on macro-F1, the metric the project is judged on: 0.51 in FP32 and 0.49 in INT8 against 0.48 for the logistic regression and 0.46 for the random forest. The forest has the highest accuracy (0.935) because it predicts N almost always on unseen records; the CNN's 0.92 is lower but comes with a far better V class. All numbers are well below the validation scores of Section 4.4. (0.67 for the CNN), which is the inter-patient gap discussed below.

5.2. Per-Class Performance

Table 18: Per-class performance on DS2.

Model	Class	Precision	Recall	F1	Beats
INT8 (QAT)	N	0.954	0.955	0.954	44 222
INT8 (QAT)	S	0.122	0.114	0.118	1 837
INT8 (QAT)	V	0.855	0.943	0.897	3 219
INT8 (QAT)	F	0.008	0.003	0.004	388
FP32	N	0.953	0.961	0.957	44 222
FP32	S	0.143	0.127	0.135	1 837
FP32	V	0.888	0.928	0.908	3 219
FP32	F	0.056	0.013	0.021	388

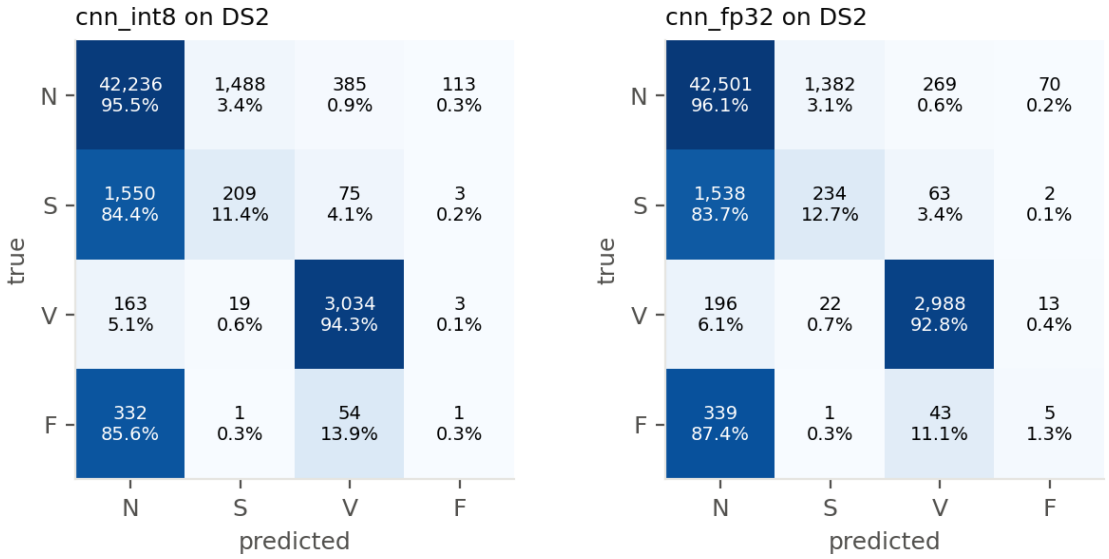


Figure 10: Confusion matrices on DS2 of the deployed INT8 model (left) and the FP32 model (right). Rows are true classes, columns predicted classes; the shading is the share of the row.

N and V are solved well. The INT8 model finds 95.5 % of normal beats and 94.3 % of ventricular beats, with a V precision of 0.86 (Figure 10). V beats have a wide, differently shaped QRS complex, which a convolution over the waveform detects on patients it has never seen.

S is the difficult class. S recall is 0.11: 1 550 of the 1 837 S beats in DS2 are called N, and 1 488 N beats are called S, so precision is 0.12 as well. On the validation records the same model found 40 % of the S beats. The explanation is in Table 23: record 232 alone holds 1 382 of the DS2 S beats, three quarters of the class. Its atrial premature beats sit in a rhythm with long pauses and slow underlying rate, so they do not look "early" in the way the training S beats do, and the network sees ordinary N beats. The only S cue available to a waveform-only model is timing relative to the neighbouring beats, and for this patient that cue points the wrong way. A single DS1 record (209) contributed most of the training S beats, so the model also learned one patient's version of S. The random forest, which gets the timing as explicit RR features, does no better (S recall 0.003); the problem is the patients, not only the features.

F is not learned. With 382 training F beats, 373 of them from record 208, the model never generalises the class: F recall is 0.003 in INT8 and 0.013 in FP32, and most F beats are called N. The 32 validation F beats were too few to guide selection. A fusion beat is by definition half normal and half ventricular, and without more patients the network has no way to know where to draw that line.

5.3. Quantization Analysis

Table 19: Effect of quantisation on DS2.

Quantity	Value
FP32 macro-F1	0.505
INT8 macro-F1 (QAT)	0.493
Difference (FP32 minus INT8)	0.012

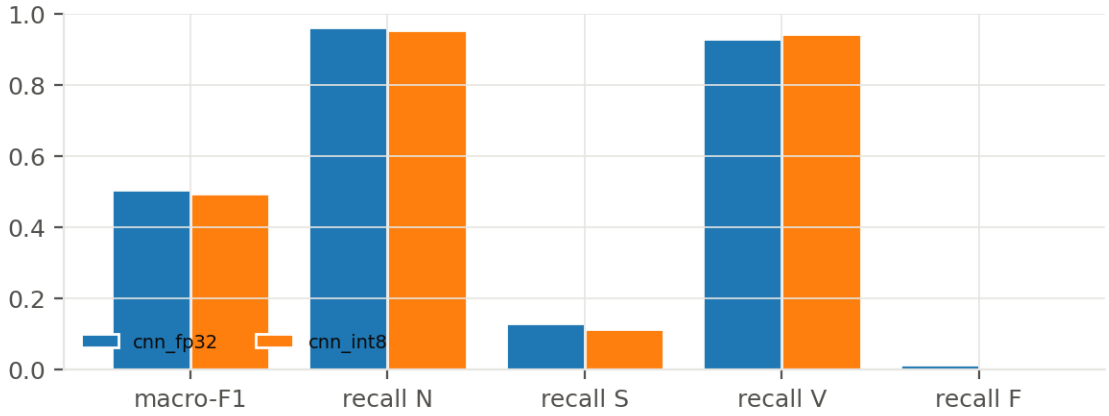


Figure 11: Macro-F1 and per-class recall on DS2 of the FP32 and the INT8 model.

The deployed INT8 model loses 0.012 macro-F1 against FP32 on DS2 (0.505 to 0.493), within the 0.03 allowed in Section 1.4. Accuracy drops by 0.005. Per class (Figure 11), V recall even rises slightly (0.928 to 0.943) while S recall falls from 0.127 to 0.114 and F from 0.013 to 0.003; the rare classes, which live on small differences in the waveform, feel the 8-bit rounding first. This small loss is the result of QAT with activation thresholds; the post-training route of Section 4.5. would have lost about 0.27 on validation.

5.4. Noise Robustness

Noise from the Noise Stress Test Database was added to the raw DS2 signals before preprocessing, at 12, 6 and 0 dB for each of the three noise types, and the frozen models were scored again (`results/noise_results.csv`). Table 20 averages the INT8 model over the three noise types, Figure 12 and Table 21 show each type.

Table 20: INT8 model on DS2 under increasing noise, averaged over the three noise types.

Condition	Accuracy	Macro-F1	F1 N	F1 S	F1 V	F1 F
Clean	0.916	0.493	0.954	0.118	0.897	0.004
12 dB	0.915	0.492	0.954	0.123	0.880	0.013
6 dB	0.908	0.481	0.950	0.117	0.844	0.013
0 dB	0.895	0.459	0.943	0.102	0.765	0.026

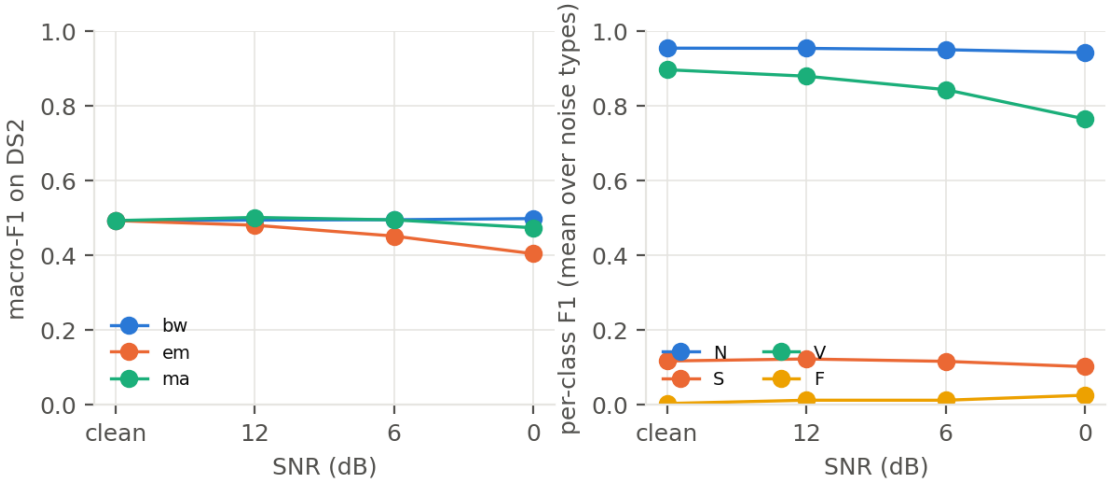


Figure 12: INT8 model on DS2 under noise. Left: macro-F1 against SNR for each noise type. Right: per-class F1 averaged over the three noise types.

Table 21: Macro-F1 on DS2 for every noise type and SNR. no_lowpass: the INT8 model with the 40 Hz filter switched off.

Noise	SNR (dB)	cnn_int8	cnn_fp32	cnn_int8_r	baseline_r	cnn_w512_	cnn_w512_
clean	clean	0.493	0.505	0.497	0.461	0.445	0.452
baseline wander	12	0.495	0.505	0.497	0.461	0.445	0.454
baseline wander	6	0.496	0.504	0.499	0.461	0.443	0.455
baseline wander	0	0.499	0.504	0.500	0.460	0.444	0.459
electrode motion	12	0.481	0.484	0.484	0.452	0.420	0.437
electrode motion	6	0.452	0.454	0.454	0.436	0.386	0.404
electrode motion	0	0.405	0.402	0.408	0.401	0.339	0.342
muscle artefact	12	0.502	0.507	0.499	0.459	0.443	0.451
muscle artefact	6	0.495	0.499	0.489	0.457	0.437	0.448
muscle artefact	0	0.474	0.476	0.461	0.452	0.413	0.429

Which classes are affected. The V class carries almost all of the loss: its F1 falls from 0.90 clean to 0.59 with electrode motion at 0 dB, while N stays above 0.91 and S and F were already near their floor. Electrode-motion artefact produces sharp transients that look like wide QRS complexes, so N beats are called V.

Degradation is gradual and depends on the noise type. Baseline wander has no effect at any level (macro-F1 0.495 to 0.499 across 12, 6 and 0 dB): the median-filter baseline removal of Section 3.4 takes it out before the network sees it. Muscle artefact costs little until 0 dB (0.502, 0.495, 0.474). Electrode motion is the hard case and degrades steadily (0.481, 0.452, 0.405). The average drop from clean to 6 dB is 0.012, far inside the 0.15 allowed.

Does preprocessing help? The baseline removal clearly does, as the `bw` rows show. The 40 Hz low-pass filter does not: with it switched off (rows `cnn_int8_no_lowpass`) the scores are the same within 0.01, and on clean data slightly higher. The network has learned to ignore high-frequency noise itself, helped by the Gaussian-noise augmentation, so the low-pass could be dropped from the device pipeline.

Robustness versus model size. The small (7 428 parameters) and large (39 300) wide-window

networks from the sweep, both FP32, score lower than the final model everywhere and degrade at least as fast; the large network is the least robust (0.339 at 0 dB electrode motion against 0.405 for the final model). More weights did not buy robustness. The random forest barely moves under noise because the RR features, taken from the annotations, are untouched by it; its waveform half is what degrades (V F1 0.87 to 0.64 at 0 dB electrode motion).

Could robustness be improved? The biggest lever is training on noisy signals: Section 3.7 planned real `nstdb` noise as an optional augmentation, which was not run in the sweep for lack of time; it would target exactly the electrode-motion case. A second lever is a simple signal-quality check on the device that rejects windows whose amplitude distribution is abnormal, rather than classifying them.

5.5. Error Analysis

Table 22: Number of beats in the main error types, INT8 model.

True to predicted	Clean DS2	Muscle artefact, 0 dB
N → S	1 488	672
S → N	1 550	1 637
V → N	163	531
V → F	3	21
F → N	332	340
F → V	54	41

Table 23: The eight DS2 records with the lowest accuracy of the INT8 model, with the true and predicted beat counts per class.

record	beats	accuracy	true N	pred N	true S	pred S	true V	pred V	true F	pred F
232	1 780	0.226	398	1 770	1 382	4	0	6	0	0
219	2 154	0.634	2 082	1 307	7	767	64	74	1	6
214	2 258	0.787	2 001	1 680	0	180	256	398	1	0
213	3 247	0.876	2 637	2 975	28	0	220	271	362	1
222	2 481	0.881	2 272	2 069	209	374	0	33	0	5
202	2 135	0.923	2 060	1 987	55	74	19	37	1	37
233	3 077	0.933	2 229	2 059	7	155	830	834	11	29
210	2 646	0.950	2 419	2 378	22	64	195	186	10	18



Figure 13: Misclassified beats of the INT8 model on clean DS2, one row per error type, four random examples each. The window is the model input (normalised amplitude, R-peak at 0 ms).

N classified as S (1 488 beats, Figure 13, first row). These are normal beats that arrive with a short or irregular gap after the previous beat, so the window shows a neighbouring R-peak early on. The model has learned "a close previous beat means S" and applies it to irregular normal rhythms: record 219, whose rhythm is atrial fibrillation, alone contributes 767 of these errors, and records 222 (374 beats called S, of which 209 are true S) and 214 (180) most of the rest (Table 23).

S classified as N (1 550 beats, second row). Almost all from record 232, where only 4 of 1 382 S beats were recognised. The beats look like normal beats with a normal QRS, which is what an atrial premature beat is, and in this patient's slow rhythm the preceding gap is not short. Nothing inside a 1.4 s window separates them from N.

V misclassified (163 as N, 19 as S). The V beats called N have a narrower QRS than the training V beats, so the width difference the model relies on is small. The opposite error is more common: in record 214, whose normal beats show left bundle branch block and are therefore wide, 142 N beats are called V. The few V beats called S are early beats with a moderate QRS width.

F beats (332 as N, 54 as V). Fusion beats are called N when the ventricular part is small and V when it is large; the model has no middle class in practice.

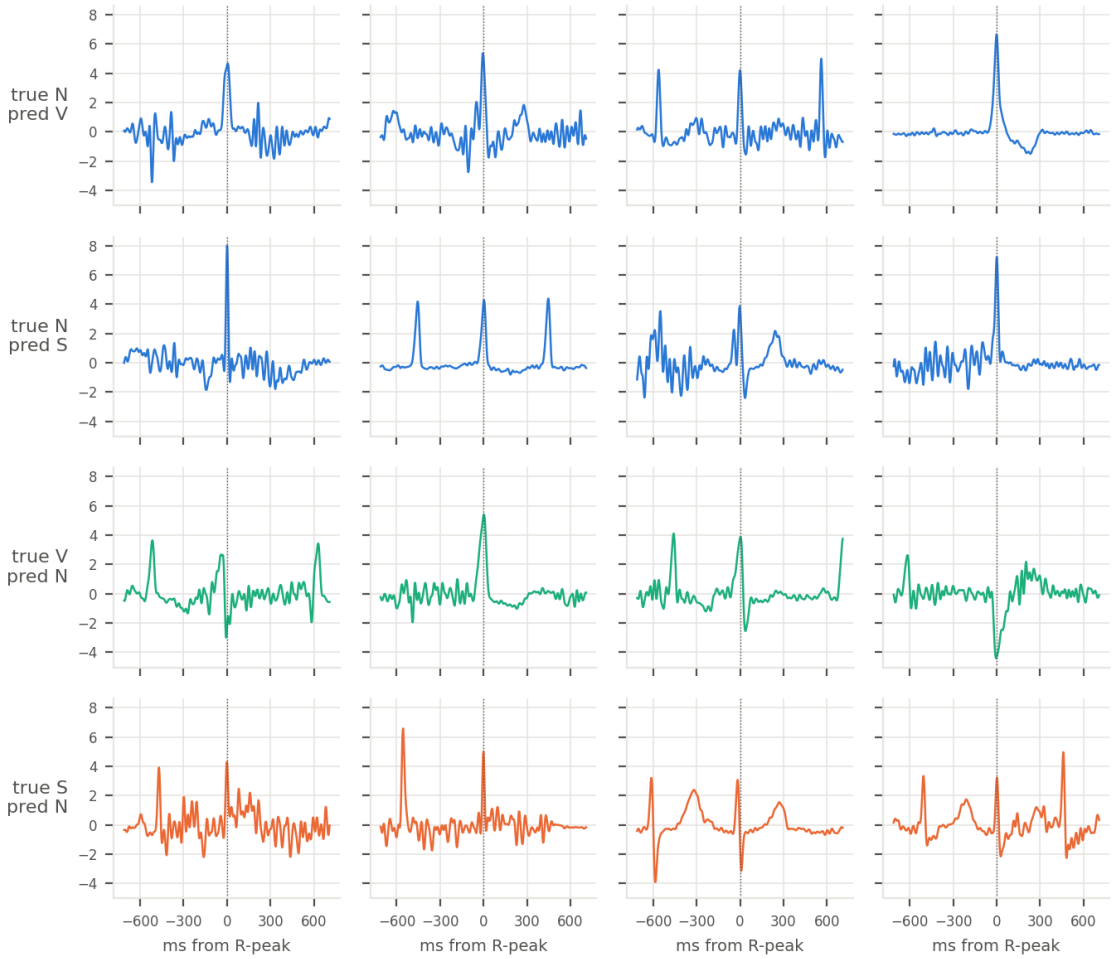


Figure 14: Beats that were classified correctly when clean but wrongly with muscle artefact at 0 dB, shown as the model saw them.

Errors under strong noise (Figure 14). With muscle artefact at 0 dB the per-beat z-score spreads the noise over the whole window, so the QRS complex loses its prominence and sharp noise peaks appear elsewhere. Normal beats then turn into V (the noise peaks read as a wide complex) and V beats into N (the real complex is buried). The P wave, which was already the weakest feature, is the first to disappear, which removes the little S information there was.

5.6. Success Criteria Review

Table 24: Success criteria of Section 1.4 reviewed on DS2.

Criterion	Target	Result on DS2	Status
Predictive performance	macro-F1 ≥ 0.70 on DS2	0.493	Not achieved
Minority class performance	recall S and F ≥ 0.50, V ≥ 0.85	S 0.11, F 0.00, V 0.94	Partially achieved
Noise robustness	macro-F1 drop clean to 6 dB ≤ 0.15	0.493 to 0.481 (drop 0.012)	Achieved
Model complexity	$\leq 100\,000$ parameters, $\leq$ 200 kB INT8	25 348 parameters, 24.6 kB	Achieved
MAX78002 compatibility	PASS on all four toolchain checks	4 of 4 PASS	Achieved
Quantisation loss	INT8 macro-F1 within 0.03 of FP32	loss 0.012	Achieved

Four of the six criteria of Section 1.4 are achieved: the model is small, it is robust to noise, it passed the whole MAX78002 toolchain with a bit-exact known-answer test, and the quantisation loss is within the limit. The predictive criteria are not achieved: macro-F1 on DS2 is 0.49 against the target of 0.70, V recall meets its target (0.94 against 0.85) but S and F recall do not (0.11 and 0.003 against 0.50). The minority-class criterion is therefore partially achieved. Section 7. discusses why the target was missed and what would change it; in short, the model does what a waveform-only single-lead classifier can do for N and V, and the S and F classes need information or data that this setting does not provide.

6. Deployment

In this course, deployment means software-level preparation for the MAX78002. No board was available, so this section reports what the Analog Devices toolchain accepted, quantised and synthesised, and nothing that would need hardware to measure.

6.1. MAX78002 Compatibility

The MAX78002 CNN accelerator executes a fixed set of operations: 1D and 2D convolutions with small kernels, max and average pooling fused into the convolution that follows, ReLU, and fully connected layers realised as 1×1 convolutions on flattened data [6]. The network of Section 4.2. uses exactly these: four fused blocks of (MaxPool1d, Conv1d, ReLU) and one Linear layer. Each block maps to one accelerator layer, so the whole network is five layers. Table 13 lists them with their shapes.

- **Input:** one channel of 512 signed 8-bit samples (the beat window divided by the clip limit 8 and scaled to $[-128, 127]$), loaded in CHW format into the data memory of processor 0.
- **Output:** four 32-bit values, one per class, read back by the Cortex-M4 core; the class is their argmax (the generated `softmax.c` turns them into probabilities when needed).
- **Parameters:** 25 348 weights and biases after BatchNorm folding, stored as INT8. The synthesiser reports 25 168 bytes of kernel memory out of 2 396 160 and 180 bytes of bias memory (Table 25), about 1 % of the chip.
- **Layer limits:** kernel lengths 5 and 3 (allowed 1 to 9), padding 2 and 1 (allowed 0 to 2), pool sizes 2 and 4 (allowed 1 to 16), stride 1, at most 64 input channels, and a flattened input of 1 024 values for the Linear layer (the ai8x limit). The largest activation is $16 \times 512 = 8\,192$ bytes, far below the 20 480 bytes per channel group of the data memory.

BatchNorm layers are used in training only. Before quantisation they are folded into the weights and biases of the preceding convolutions (`ai8x.fuse_bn_layers`), which is the standard procedure for this accelerator [5]. Dropout is the identity at inference. Neither appears in the synthesised network.

6.2. MAX78002 Toolchain

The two Analog Devices repositories were used as supplied [6], [7], cloned by `make toolchain`:

1. **Preparation (ai8x-training).** The network is built from the `ai8x` layer wrappers (`FusedMaxPoolConv1dReLU`, `FusedConv1dReLU`, `Linear`) in `src/ecg/models/ai8x_cnn.py`. The wrappers check every layer against the accelerator limits when the model is built, behave like ordinary PyTorch layers in FP32, and implement quantisation-aware training and INT8 simulation. Training uses the project's own loop with these layers; the files in `max78002/model_configuration/` register the same network and dataset in the official `train.py` for anyone who prefers that route.
2. **Quantisation-aware training.** After the FP32 model is chosen, BatchNorm is folded, the activation range of every layer is measured on training data and turned into a power-of-two scale per layer (ai8x's activation thresholds), and the network is fine-tuned for 15 epochs with 8-bit weights and clamped activations (`ai8x.initiate_qat`). Section 4.5. shows why this step is necessary.
3. **Quantisation (ai8x-synthesis).** `quantize.py -device MAX78002` converts the checkpoint to integer weights, biases and output shifts. It was run on the FP32 weights (post-training quantisation) and on the QAT weights. Logs and both INT8 checkpoints are in `max78002/quantization_output/`.
4. **Synthesis and C code.** `ai8xize.py -device MAX78002` reads the INT8 checkpoint, the network description `max78002/model_configuration/ecg-max78002.yaml` (generated from `configs/model.yaml`) and one sample beat, and writes the C project in `max78002/synthesis_ou` `max78002/`: `cnn.c` and `cnn.h` (accelerator configuration, weight loading, inference), `weights.h`, `sampledata.h`, `sampleoutput.h`, `main.c` with a known-answer test, `softmax.c` and a Makefile for the MSDK.

The first synthesis attempt failed with "Layer 0 output is overwriting input": the network description placed the first layer's output at data-memory offset 0, where the input is loaded. Alternating the output offsets between 0x8000 and 0x0000 fixed it. The second attempt failed because the architecture name in the checkpoint did not match the YAML; both now use `ai87ecgnet`. No change to the network itself was needed at any point.

6.3. Deployment Evidence

Table 25: Deployment evidence from the MAX78002 toolchain (software only, no board).

Requirement	Result	Evidence
Model accepted by MAX78002 toolchain	PASS	ai8x layer checks, quantize.py and ai8xize.py read the model without error
Quantization completed	PASS	quantize.py log and the INT8 checkpoint
Synthesis completed	PASS	ai8xize.py exit code and log.txt
C/device files generated	PASS	cnn.c, cnn.h, weights.h, main.c, sampledata.h, sampleoutput.h, softmax.c, Makefile
Weight memory requirement	25 168 bytes of 2 396 160	RESOURCE USAGE in log.txt
Data memory requirement	8 192 bytes (largest activation) of 1 310 720	largest layer input or output; the MAX78002 has 1 280 KiB of data memory
Known-answer test: Python INT8 output equals synthesiser output	PASS	sampleoutput.h versus the Python INT8 simulation of the same beat

The last row is a check that goes beyond the toolchain's own exit codes. The synthesiser computes the network output for the sample beat with its bit-exact model of the accelerator and writes it into `sampleoutput.h`, where `main.c` compares it with the real hardware output. The same INT8 checkpoint was run on the same beat in the Python integer simulation (`ai8x` with `simulate=True`, the mode behind every INT8 score in Section 5.). The four output values agree exactly, so the INT8 results reported in this document describe the arithmetic the generated code will execute.

6.4. Hardware Limitations

Table 26: What this project verified, and what it could not.

Software verified	Hardware verified
The network uses only accelerator operations and passes the layer checks of `ai8x` and `ai8xize.py`	Nothing. No MAX78002 board was used
INT8 accuracy, macro-F1 and per-class scores on DS2, in the bit-exact integer simulation	Inference latency: not measured
Generated C project with weights, sample input and known-answer test	Power and energy per inference: not measured
Weight memory (25 168 bytes) and data memory from the synthesiser	Accuracy on the device: not measured
Python simulation and synthesiser agree on the sample output	Real-time behaviour with a live ECG front end: not measured

The numbers in this report are therefore estimates of what the device would do, not measurements. The synthesiser's operation count (967 680 operations per beat, of which 942 080 multiply-

accumulates) gives an idea of the workload, but no latency is claimed from it.

6.5. Proposed Hardware Validation

With a MAX78002 evaluation kit the following steps would close the gap, in this order:

1. **Load the generated model.** Build `max78002/synthesis_output/ecg-max78002/` with the MSDK and flash it. `main.c` loads the weights, runs the sample beat and compares the output with `sampleoutput.h`; a pass shows that weights, layer configuration and data layout are correct on silicon.
2. **Identical inputs.** Replace the built-in sample with DS2 beats streamed over the serial port, already preprocessed exactly as in Section 3. and scaled to $[-128, 127]$ with `src/ecg/deployment/sample_input.py`, so that the device and Python see the same bytes.
3. **Compare outputs.** Read back the four output values per beat and compare them with the Python INT8 simulation. They must match exactly; then the device's DS2 confusion matrix equals Table 18 by construction.
4. **Intermediate consistency.** If any output differs, dump the layer outputs with `ai8xize.py -log-intermediate` and read the corresponding data-memory regions from the device, to find the first layer that diverges.
5. **Inference time.** Use the accelerator's cycle counter and a GPIO toggle around `cnn_start` and the completion interrupt to measure latency per beat, with the clock settings of `cnn.c`.
6. **Energy.** Use the evaluation kit's power monitor to measure current during repeated inference, and report energy per beat separately for the accelerator and the Cortex-M4.
7. **Live signal.** Finally, feed the ECG front end with a signal generator or a recorded MIT-BIH record and check that the on-device preprocessing (median filters, normalisation) reproduces the Python arrays within rounding.

7. Final Discussion

What the data taught us. Three things shaped the whole project. First, the rare classes are not spread over patients: most F beats come from one training record and three quarters of the DS2 S beats from one test record (232), so "learning S" and "learning F" largely means learning one person, and the inter-patient score measures how well that transfers to another person, which is poorly. Second, the information that separates an S beat from an N beat is timing, not shape; the best single change in the project was widening the input window so that the network could see the neighbouring beats (validation S recall doubled), and even then timing failed on a patient whose rhythm does not follow the usual pattern. Third, the choice of validation records matters as much as the model: with the noisiest record and both bundle-branch-block records in validation, model selection chose an almost untrained network; after the revision of Section 3.1. the sweep became informative.

The biggest source of error is the S class, in both directions: 1 550 S beats called N and 1 488 N beats called S on DS2. One patient (232) supplies most of the first group; an atrial-fibrillation patient (219), whose normal beats arrive irregularly, half of the second. Together they cost more macro-F1 than everything else combined, and they explain the gap between validation (0.67) and DS2 (0.49). N and V, the classes with a shape signature, are handled well on unseen patients.

Was the highest-accuracy model also the best Edge-AI model? Not quite, and the difference is instructive. The random forest has the highest DS2 accuracy (0.935) but the lowest macro-F1, because it predicts N almost always and finds 0.3 % of the S beats. The plain cross-entropy CNN had the highest validation accuracy but a poor S recall. The final model trades about 1.5 points of accuracy for V recall of 0.94 and whatever S recall the data allowed, and it is the one that runs on the chip: 25 kB of weights, five accelerator layers, no operator on the CPU. Among the CNNs, size did not matter: the 7 428-parameter version was within 0.07 of the final on validation and would have been the right choice if memory had been tight.

How much performance was lost after quantisation? 0.012 macro-F1 on DS2, after quantisation-aware training with per-layer activation scales. Without that preparation the loss would have been about 0.27: the FP32 network uses activation ranges the 8-bit clamp cuts off. The lesson is that for this accelerator QAT is not an optimisation but a requirement, and that the activation-threshold step of the toolchain is the part that does the work.

How sensitive was the system to noise? Less than expected for baseline wander and muscle artefact, which the median filter and the network absorb (drop at 6 dB below 0.02 on average), and clearly sensitive to electrode motion, which imitates ventricular beats (macro-F1 0.405 at 0 dB, V F1 from 0.90 to 0.59). The low-pass filter turned out to be unnecessary. Robustness came from the preprocessing and the augmentation, not from model size; the largest network was the least robust.

What compromises did the hardware force? The operator set (no attention, no recurrence, no normalisation at inference) was not a real limit for this problem; a small 1D CNN is the right tool anyway. Two constraints did shape decisions: the fully connected layer may take at most 1 024 inputs, which fixed the pooling schedule of the wide-window model, and the 8-bit activation range, which forced QAT and the activation scaling. The fixed window size means the model cannot use variable-length context such as the last ten RR intervals, which is exactly what would help the S class; adding those as a second small input is possible on the accelerator but was not attempted.

What we would change with more time. In order of expected gain: (1) give the network the RR features explicitly, as a second input branch merged before the output layer, so that timing does not have to be read from the waveform; (2) train with real `nstdb` noise at random SNR, which targets the electrode-motion weakness directly; (3) evaluate with record-wise cross-validation inside DS1 instead of one validation split, to make model selection less dependent on five records; (4) a per-patient adaptation step on the device, using the first minute of a recording to calibrate the N class, which published inter-patient work shows to be the largest single improvement for S; and (5) a real board, to turn the software-verified claims of Section 6. into measurements.

8. Limitations

Limited dataset size. MIT-BIH has 47 subjects, and after the paced records are removed the model learns from 17 people and is tested on 22. About 100 000 beats sound like a lot, but beats from one person are near duplicates of each other, so the effective sample size is the number of people, not of beats. Every result in this report rests on a handful of patients per class.

Class imbalance. The training set holds 36 112 N beats and 382 F beats, and the rare classes come from very few records (most S from 209, most F from 208). Balancing by sampling helped the V class and gave some S recall, but no balancing method can add patients. The F class in particular is not learned, and the 32 validation F beats could not guide selection.

Differences between patients. The inter-patient gap is the main finding: validation macro-F1 0.67, DS2 0.49, with one DS2 patient (232) accounting for most of the S errors. The model's notion of "early" is learned from a few rhythms and does not transfer to a slow rhythm with pauses. Beat shape, amplitude, heart rate and noise level all differ between people, and the fixed window cannot adapt to them. Conclusions from 22 test records are also fragile: moving one record would change several scores by more than the quantisation loss.

Use of historical ECG datasets. The recordings date from 1975 to 1979, were made in a hospital with chest electrodes on a Holter recorder, and were selected to contain interesting arrhythmias. Equipment, patient population and prevalence of each class differ from any present-day use. The annotations are expert consensus from that time and contain a small amount of label noise, especially in the noisy records that were kept.

Absence of prospective clinical testing. No prospective data, no clinical workflow and no clinician were involved. The model is not a medical device and nothing in this report is a claim of clinical validity. The class definitions follow the AAMI grouping, but a clinical system would need an unclassifiable output, a signal-quality output and a validated alarm policy, none of which exist here.

Lack of hardware testing. All deployment results are software-level: the toolchain accepted, quantised and synthesised the network, and the Python INT8 simulation agrees with the synthesiser's known-answer output. No MAX78002 board was available, so inference time, power, energy per beat, on-device accuracy and real-time behaviour are unmeasured (Section 6.4.). Section 6.5. describes how to measure them.

Possible domain shift to wearable ECG. A wearable records a different lead, with dry electrodes, at a different sampling rate and resolution, with far more motion artefact, and without expert R-peak annotations. This project used the annotated R-peaks as input, so a device would additionally need a beat detector whose errors propagate into the classifier. The noise experiment used recorded noise added to hospital ECG, which is a reasonable but incomplete stand-in; electrode-motion artefact, the kind a wearable produces most, is exactly the case where the model degrades. Resampling, re-annotation and new recordings would all be needed before the reported numbers mean anything for a wearable device.

References

[1] G. B. Moody and R. G. Mark, 'The impact of the MIT-BIH arrhythmia database,' *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001. DOI: 10.1109/51.932724.

[2] G. B. Moody, W. E. Muldrow and R. G. Mark, 'A noise stress test for arrhythmia detectors,' in *Computers in Cardiology*, vol. 11, 1984, pp. 381–384.

[3] P. de Chazal, M. O'Dwyer and R. B. Reilly, 'Automatic classification of heartbeats using ECG morphology and heartbeat interval features,' *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004. DOI: 10.1109/TBME.2004.827359.

[4] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, 'Focal loss for dense object detection,' *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2980–2988, 2017.

[5] B. Jacob, S. Kligys, B. Chen *et al.*, 'Quantization and training of neural networks for efficient integer-arithmetic-only inference,' *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, 2018.

[6] Analog Devices, *Ai8x-synthesis: Quantization and synthesis for the MAX78000 and MAX78002*, https://github.com/analogdevicesinc/ai8x-synthesis, Accessed November 2026, 2024.

[7] Analog Devices, *Ai8x-training: Model training for the MAX78000 and MAX78002*, https://github.com/analogdevicesinc/ai8x-training, Accessed November 2026, 2024.